Индивидуальное задание для практики 3-4.1

Класс Functor

```python
class Functor:
    def __init__(self, a, b, c):  # Конструктор
        self.__a = a  # Закрытый атрибут
        self.__b = b  # Закрытый атрибут
        self.__c = c  # Закрытый атрибут
        self.__hist = []

    def calc(self, x, y):  # Обычный метод
        result = (self.__a * x) / (self.__b * y) + self.__c
        self.__hist.append((x, y, result))
        return result

    def history(self):  # Возвращает историю вычислений в виде кортежа
        return tuple(self.__hist)

    def max_result(self):  # Вычисляет максимальный результат
        current_max = None
        for h in self.__hist:
            if current_max == None or current_max < h[-1]:
                current_max = h[-1]  # Берем последнее значение из кортежа
        return current_max

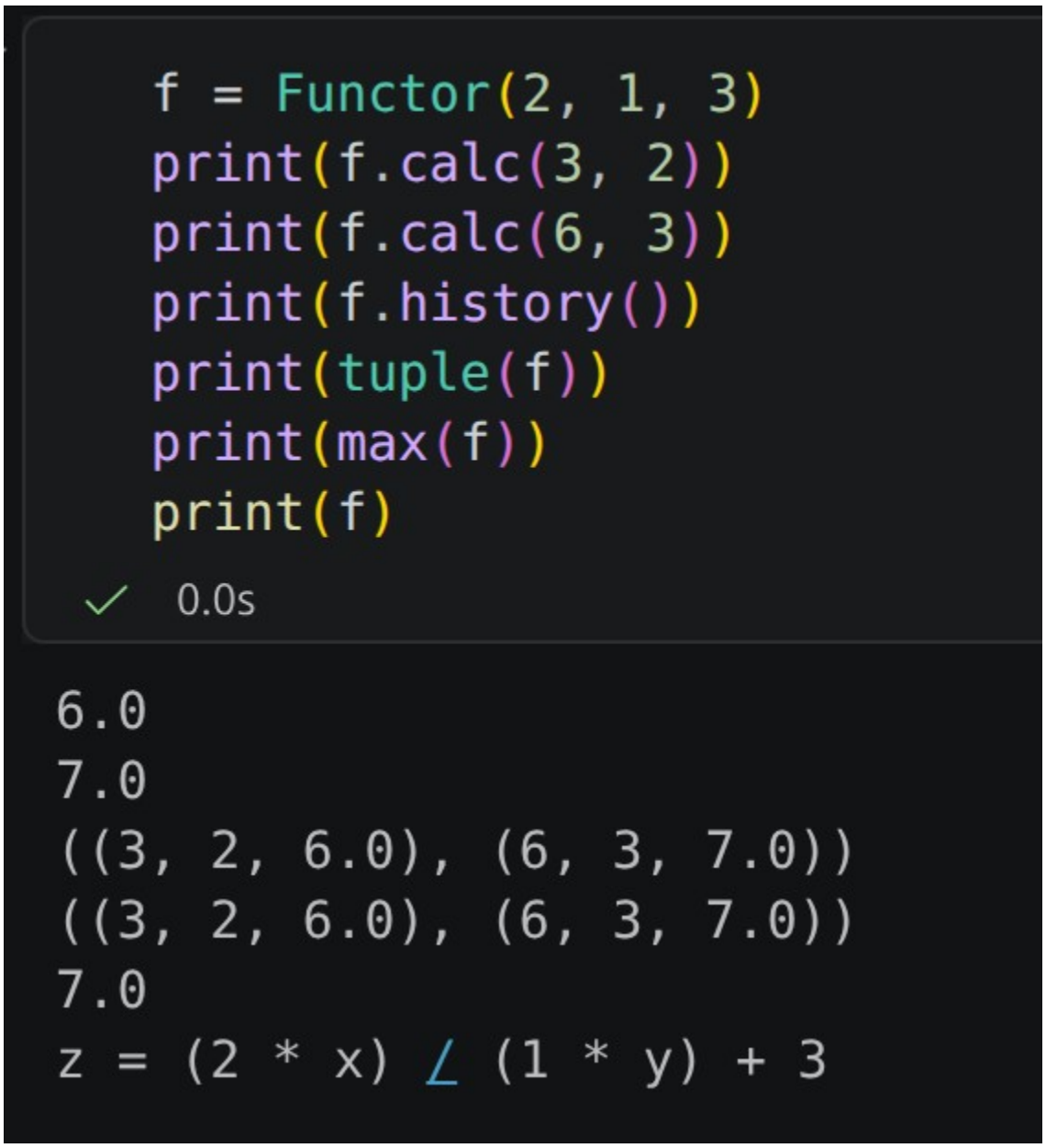
    def __iter__(self):  # Позволяет итерироваться по внутреннему списку
        return iter(self.__hist)

    def __str__(self):  # Выводит описание в нужном формате
        return f"z = ({self.__a} * x) / ({self.__b} * y) + {self.__c}"


_original_max = max  # Сохраняем функцию


def max(*args, **kwargs):
    # Проверяем что у объекта есть атрибут max_result
    if len(args) == 1 and hasattr(args[0], "max_result"):
        return args[0].max_result()
    return _original_max(*args, **kwargs)  # Иначе стандартная max
```

Printscreen-ы с результатами



Передаем в Functor значения a, b, c, далее вычисляем резуьтаты для разных x и y.

Вызываем историю и проверяем tuple.

Находим максимальный результат и смотрим строкове представление объекта.

Класс NewImageDraw

```python
from PIL import ImageDraw


class NewImageDraw:
    def __init__(self, img):
        self.img = img  # Исходное изображение

    def triangle(self, xys: tuple, fill):
        length = xys[2]  # Длина равностороннего треугольника
        if length <= 0:
            raise ValueError("Value must be positive")
        x = (length**2 - (length / 2) ** 2) ** 0.5  # Теорема пифагора
        draw = ImageDraw.Draw(self.img)
        # По x отнимаем катет, по y смещаем на половину длинны стороны
        draw.polygon(
            [
                (xys[0] - int(x), xys[1] + length // 2),
                (xys[0] - int(x), xys[1] - length // 2),
                (xys[0], xys[1]),
            ],
            fill=fill,
        )
```

Координаты двух других вершин вычисляются по теореме пифагора.

Пример работы программы

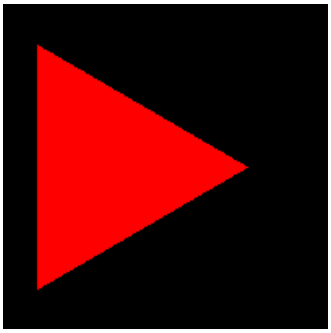
```
from PIL import Image

im = Image.open("test.jpg")
dr = NewImageDraw(im)
dr.triangle((150, 100, 150), (255, 0, 0))
# im.show()
im.save("image.bmp")
```

Создаем объект NewImageDraw, передаем изображением и вызываем метод triangle, в качестве аргументов передаем кортеж в котором первые два значения это координаты правой вершины, третье значение – длинна стороны. Остальные координаты вершин вычисляются внутри метода.

Вторым аргументом передаем кортеж RGB с интесивностью цветов, в данном случае красный.

Результат



Класс Unit

```python
class Unit:
    def __init__(self, name: str, unit: str):
        self._name = name
        self._unit = unit
        self._coefficients = {}

    # Возвращает название с заглавной буквы
    def name(self):
        return self._name[0].capitalize() + self._name[1:]

    def symbol(self):
        return self._unit.lower()  # Возвращает обозначение в нижнем регистре

    def set(self, other_unit, coefficient):
        # Проверка что передаваемый класс принадлежит Unit
        if isinstance(other_unit, Unit):
            self._coefficients[other_unit.symbol()] = coefficient
        else:  # Иначе ошибка
            raise TypeError("other_unit must be Unit")

    def remove(self, other_unit):
        # Проверка что передаваемый класс принадлежит Unit
        if isinstance(other_unit, Unit):
            del self._coefficients[other_unit.symbol()]
        else:  # Иначе ошибка
            raise TypeError("other_unit must be Unit")

    def coefficients(self):
        # Возваращаем коэффициенты в виде кортежа
        return tuple(self._coefficients.items())

    def __iter__(self):
        # Итератор для tuple()
        return iter(self._coefficients.items())

    def __str__(self):  # Выводит описание в нужном формате
        return self._unit
```

Класс Measure

```python
class Measure:
    def __init__(self, unit: Unit, value):
        self._unit = unit
        self._value = value

    def value(self):
        return self._value

    def convert(self, unit: Unit):
        # Достаем коэффициент под нужный символ
        coefficient = self._unit._coefficients.get(unit.symbol())
        if coefficient == None:
            return None
        # Возвращаем новый объект с пересчитанным значением
        return Measure(unit, self._value * coefficient)

    def __add__(self, other_measure):
        try:
            curr = other_measure(self._unit)
            return Measure(self._unit, self.value() + curr.value())
        except Exception:
            return None

    def __call__(self, *args, **kwds):
        coefficient = self._unit._coefficients[args[0].symbol()]
        if coefficient == None:
            return None
        return Measure(args[0], self._value * coefficient)

    def __str__(self):  # Выводит описание в нужном формате
        return str(int(self._value)) + str(self._unit)
```

Оба класса лежат в solution.py

Класс Utin описывает систему измерения, а класс Measure само измерение, позволяет переводить в различные системы измерения и поддерживает сложение.

Результаты

```
from solution import Measure, Unit
```

```
meter = Unit("метр", "м")
kilometer = Unit("километр", "км")
meter.set(kilometer, 1 / 1000)
meters = Measure(meter, 10000)
print(meters.value())
kilometers = meters.convert(kilometer)
print(meters, kilometers)
kilometers = meters(kilometer)
print(meters, kilometers)
print(meters + kilometers)
kilometer.set(meter, 1000)
print(meters + kilometers)
```

```
10000
10000м 10км
10000м 10км
None
20000м
```

```
meter = Unit("метр", "м")
print(meter.name())
print(meter.symbol())
kilometer = Unit("километр", "км")
meter.set(kilometer, 1 / 1000)
print(meter.coefficients())
print(tuple(meter))
meter.remove(kilometer)
print(meter.coefficients())
print(tuple(meter))
print(meter)
```

```
Метр
м
(('км', 0.001),)
(('км', 0.001),)
()
()
м
```